

Delta Compression for a Fast Distributed Archive of Genomic Data *

ABSTRACT

It is conceivable that in the future everyone's genome will be sequenced in order to provide personalized medicine. Specifically, one's genome will be stored along with her medical records and will need to be transported between sequencing centers, storage depots, hospitals, and research facilities efficiently and securely. Current genome distribution methods rely upon a traditional client server model with per file compression—a scalable distributed solution is needed to meet the needs of tomorrow's personalized medicine. We approach this problem by examining ways of reducing the amount of data that is sent. Specifically, we show that genomic data has exploitable redundancies by reducing a 3GB genome to a mere 11MB of deltas. This work offers a preliminary investigation into the use of delta compression for fast genomic data transfer, and examines the applicability of existing data de-duplication schemes from operating systems research to efficient genomic data transfer.

Categories and Subject Descriptors

D.4.3 [File System Management]: Distributed file systems

General Terms

Design

Keywords

genomic data transfer, genome compression

1. INTRODUCTION

As the price of human genome sequencing plummets [22], it is conceivable that in the future everyone's genome will be sequenced to provide personalized medicine. Specifically, an individual's sequenced genome will be stored along with her medical records and will need to be transported quickly and

securely between sequencing centers, storage depots, hospitals, and research facilities. Such an infrastructure is required for developing health fields. Personalized medicine aims to use a patient's sequenced genome to improve the diagnostic accuracy of disease and disease susceptibility, and potentially provide individualized treatments [10]. Given the Internet infrastructure, current genomic distribution techniques will not scale to meet the needs of such a futuristic healthcare system. This paper investigates a content specific approach to genomic data distribution: can inherent data redundancies be efficiently exploited to reduce the amount of data that is sent, and reduce the overall end-to-end transmission duration?

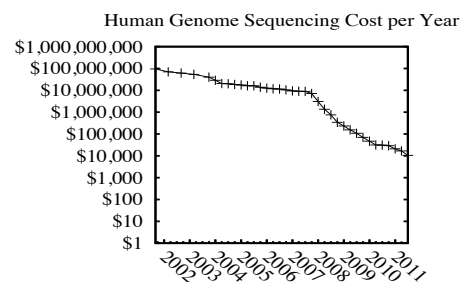


Figure 1: Sequencing costs are plummeting faster than Moore's law. Source: National Human Genome Research Institute.

This paper provides a preliminary investigation to a loss-less content specific approach to genomic data distribution that reduces aggregate data transmission by exploiting the redundancies across individual genomes, instead of focusing on compression for a single genome. Such a de-duplication strategy for large-scale genomic data storage has recently been proposed [4, 11, 24] and promises compression rates of up to two orders of magnitude. Delta compression approaches achieve high compression rates by only storing the differences between a genome and a reference. Unlike existing genomic delta compression approaches [11, 24], these techniques do not require alignment maps or sequence specific metadata: they only operate on pairs of raw genomes, analogous to how Unix `diff` operates on pairs of source code files. This paper augments this work by applying these methods to data dissemination. The author is unaware of

*Final paper for Professor Scott Brandt's CMPS221: Advanced Operating Systems class, fall 2011.

any projects that aim to efficiently transfer genomic data using a content-specific approach.

2. BACKGROUND

We identify the following goals for efficient genomic data transfer: 1) quantify the amount of redundancy between different genomes and develop a quick way of identifying similar genomes, 2) find effective windowing algorithms (see Appendix A for definitions), and 3) find efficient delta compression algorithms. This research is preliminary in that it: 1) uses and evaluates existing tools traditionally developed in operating systems, and 2) aims to contribute towards the development of future genome specific tools.

2.1 Genomic Distribution Problem

Current genomic distribution techniques, such as those used by the 1000 genomes project [7] and the UC Santa Cruz genome browser [13], will not scale to the demands imposed by personalized medicine: they rely upon a centralized client server model (FTP) that is content agnostic and separates compression from distribution. A human genome stored uncompressed in the FASTA file format is approximately 3.0GB, or 892MB using `gzip` compression (a popular compression technique). If every American had her genome sequenced today this would total over 927 petabytes of uncompressed data, or 275 petabytes of compressed data. Additional metadata and annotations such as those used in the popular BAM [15] file format can increase the size of each genome by over two orders of magnitude, or dozens of exabytes in this scenario. Disregarding the file storage problems of securely and reliably storing this amount of data, the current Internet infrastructure cannot support data transfers of this size, which may occur during inter-genome sequence center transfers. Genomes may be sequenced multiple times to increase resolution, incurring additional storage costs and data redundancy. From the bioinformatics perspective, genome sequencing costs are decreasing by at least an order of magnitude every five years [22], with a \$1000 genome rapidly approaching. This rate is orders of magnitude faster than the traditional “Moore’s Law” trends seen in transistor density and network capacity. It is clear that an alternative distribution mechanism is needed for cheap genome sequencing to become ubiquitous, and we believe that the network will become an increasing cost in genome research.

2.2 Approaches

Bioinformatics researchers are already experiencing the genomic big data problem, and are actively investigating methods for decreasing transmission duration. For example, the Bio-mirror [1] distribution site is currently testing the GridFTP [3] tool which augments FTP and supports a UDP-based transport protocol, UDT [9], to increase data transfer rates. These methods attack the big data problem from the transport layer and are not content specific: they are effective across all types of data transfer and are orthogonal to a content-specific approach [20]. Although provisioned paths and UDP-based protocols may provide temporary relief for data transfers between specific sequencing centers, it cannot be applied for general many-to-many transfers, or transfers between hosts on the network edges.

3. RELATED WORK

There is a great body of work for efficient genome compression and general data transfer between the bioinformatics and operating systems fields. This paper examines the effectiveness of traditional tools from operating systems research on genomic data, and compares some preliminary results against published results in bioinformatics. One goal of this paper is to help bridge the gap between the two fields, and serves as a first step of an interdisciplinary approach to designing an efficient system for genomic data distribution.

3.1 Bioinformatics Related Work

Christley et. al. [5] identify the eminent genomic big data problem and suggest a content specific, delta compression approach. They develop DNAZip and compress James Watson’s genome to a mere 4MB, using several methods for efficiently encoding SNP instructions. They make the additional observation that SNP maps (the sequence of instructions that build a target genome from a source genome) are also common among individuals, and they allow instructions to index into a common SNP database. Although this approach achieves significant compression, it uses a single reference and requires an existing SNP map (≈ 1.2 GB). Instead, we explore the use of data from multiple references and do not use an SNP map. Indeed, we measure the effectiveness of existing data differencing algorithms to build these mappings.

Brandon et. al. [4] also take a delta compression approach to genome storage and identify that the current data dissemination techniques are unscalable. They observe that the reference may not necessarily be a human genome and propose a “consensus” genome to increase compression. They focus on algorithms and data structures for mitochondrial genome sequences which have higher variability than chromosomal sequences. They propose an instruction set consisting of insert, delete and substitution operators with position, length and value operands. They note that a position dictionary can increase compression since the position values are typically a small subset of every possible position, depending on the population. They examine the effectiveness of Huffman, Golomb, and Elias codes [6] for compressing instructions and achieve compression rates of up to two orders of magnitude for mitochondrial DNA. The authors do not detail how the SNPs are calculated, but rely upon an existing mapping database. They also do not provide compression results for an entire human genome, although they achieve up to 3 orders of magnitude when applying these approaches to chromosome 22 of the hg18 human genome.

Heath et. al. [11] build on the previously mentioned work and propose a delta compression technique that supports indexing. With this approach, a compressed genome can be stored on disk and still provide efficient string searching. The authors make the explicit tradeoff of slightly worse compression for better indexing, hence this work mainly applicable to data storage and genome search. The instruction set used includes insert, delete, replacement, insert after replacement, and delete after replacement. Position operands are absolute and two separate Huffman codes are used for positions and value (base pairs) operands. Each instruction contains two bits to indicate whether or not the position or value operands are Huffman encoded, to efficiently

handle cases where the original value is more compact than the code. As with the previous work, they evaluate their approach using mitochondrial DNA and rely upon existing SNP maps and a single reference genome. They achieve up to two orders of magnitude compression and observe that a few positions and a few instructions occur with great frequency.

Fritz. et. al. [8] propose a delta compression approach that supports a controlled loss of precision to increase storage efficiency. Like the previous approaches, they rely upon existing sequence alignment map metadata to guide compression. Indeed, the authors suggest that additional compression techniques are needed to handle unaligned reads.

Wang. et. al. [24] provide a delta compression technique, GRS, that does not rely upon precomputed SNPs, and is perhaps the closest approach to the one examined in this paper. They propose a variable length windowing algorithm that splits the source and target genome into n windows, based on a function that minimizes the difference in nucleotide frequency counts. To compute the difference, the authors modified Unix `diff` and applied Huffman coding to the patch file. This approach was evaluated against two human genome sequences from the KOREF data set [2], two rice genomes, and an *A. thaliana* genome. They achieved more than two orders of magnitude of compression with the human genome, reducing the 2986.8MB target to a mere 18.8MB. The authors note that SNPs may be unavailable for many *de novo* genomes and demonstrate that high compression can be achieved without this constraint. We claim that an effective delta compression strategy that does not rely upon SNP maps such as the one proposed in [24] enables flexibility in distributed file system scenarios where the reference genome may not be readily available, or the compression may need to occur on-demand based on data that is already present on the sender and the receiver. We augment the work in [24] and evaluate a window based delta compression strategy on the KOREF dataset and achieve slightly better results (11MB), further validating this approach to genomic data transfer.

3.2 Operating Systems Related Work

Manber [16] presents a tool called *sif* that allows users to quickly find similar files in a file system, using a technique known as approximate fingerprints. This approach avoids the high computational complexity of performing all pairwise comparisons of a set of files in order to find their similarity. A set of fingerprints is computed for each file and is stored in a database that maps files to fingerprints, as well as the inverted index. Files that share a large number of overlapping fingerprints are deemed similar as they contain common substrings with high probability.

The author identifies the “synchronization” problem when comparing the similarity between files: one cannot compare fixed length strings at fixed positions since any single insertion or deletion before or within this range can give a different fingerprint. To solve the synchronization problem the author proposes “anchors”, or substrings that appear within the file at relatively uniform intervals. Checksums are computed over the fixed length window immediately following an anchor, and are stored in the fingerprint database. Anchor

strings can either be picked by the user or selected automatically by calculating a special hash value over all fixed length substrings and selecting certain substrings based on this hash (i.e. hashes whose last k bits are 0). The second method is more general in that the user does not need to find a good anchor string, however it relies upon a quick method for calculating the hashes of every substring: he uses a rolling window where each new byte corresponds to a few arithmetic operations to generate the next hash. We believe that the author’s approach for quickly identifying similarity across a large set of files is necessary for genomic distribution. One can envision a policy such that if Alice wishes to send a target genome T to Bob, the choice of a reference genome S should be: 1) common to Bob and Alice 2) the “most similar” of the common genomes to T , using the intuition that genomes that are “most similar” lead to the best delta compression.

Muthitacharoen et. al. [17] introduce LBFS, a file system designed for low-bandwidth networks. Using a similar approach to the one proposed by this paper, the authors exploit redundancies between files to avoid sending duplicate data across the network. A client maintains a large cache of file chunks (windows) of files that it has previously received, and chunks are efficiently indexed by the first 64 bits of the chunk’s SHA-1 hash, using a B-tree data structure. The client and server use a modified version of NFS to negotiate sending only the necessary chunks. In this system, if Bob retrieves a file T from Alice, then 1) Bob sends Alice a request for T , 2) Alice responds with a summary of all of the chunks for T and their respective hashes, 3) Bob compares these hashes with those in its client database and individually requests the chunks belonging to T that are missing from his database. To avoid pathological cases, the authors set a minimum chunk size to 2KB and maximum size of 64KB. To avoid the file synchronization problem identified by Manber [16], the authors use a similar windowing algorithm and hash every 48-byte substring in the file and examine the last 13 bits of the hash for a particular pattern (i.e. all 0s) to determine “breakpoints,” or window boundaries. In order to compute the fixed length hashes efficiently, they use the Rabin fingerprint [21] algorithm which operates over a rolling window and computes several arithmetic operations for each new hash. The use of a rolling window and a Rabin fingerprint to find chunk boundaries, with a SHA-1 hash to identify the chunk, is sometimes known as *content defined chunking*.

Pucha et. al. [19] propose SET, an approach for identifying similar files in a peer-to-peer network in order to speed up data transfer by increasing the number of file sources. The authors download a large sample of multimedia files from popular BitTorrent sites and quantify the large amount of redundancy across files of the same type. SET augments existing peer-to-peer applications such as BitTorrent by utilizing the application’s underlying DHT (distributed hash table) to store additional metadata. Their approach enables a constant number of DHT lookups to locate similar files that contain identical chunks. The authors observe that file transfers are frequently unable to saturate the available bandwidth and demonstrate that additional sources can decrease file transfer duration. SET uses handprinting to measure similarity: each file F is divided into n chunks using

Rabin fingerprinting to determine window boundaries (as in LBFS) and each chunk is SHA-1 hashed (h_i) and sorted lexicographically such that the first k hashes compose the handprint. The $\{(h_i, F) : 1 \leq i \leq k\}$ mappings from the handprint are inserted into the DHT so that a peer requesting a particular file can quickly find other files that contain the same chunks, and consequently discover more peers. Their evaluation demonstrates the effectiveness of this technique in low-bandwidth, or high bandwidth delay product (BDP) network configurations.

Jain et. al. [12] propose TAPER, a system for efficient file system replica synchronization across distributed servers. They use a multi-phase approach that eliminates data redundancy at different granularities, enabling clients to explicitly trade CPU utilization for bandwidth utilization. Their tool implements an interface similar to `rsync`, and synchronizes two separate directory trees on separate hosts. The first phase of redundancy elimination uses a hierarchical hash tree, similar to a Merkle tree, that encodes both the content and the structure of the file system. This prevents the source from sending all identical files and directories to the target. The second phase uses content defined chunks (CDCs), windowed by Rabin fingerprints with an expected chunk size of 2KB and a maximum size of 64KB—the authors note that CDCs are not suitable when small-sized updates are spread throughout the data. To fill the “holes” of unmatched chunks, in the third phase TAPER computes a rolling checksum with Rabin fingerprints over every unmatched 700 byte window, where the target requests the unmatched chunks from the source. In the last phase, the source delta encodes the remaining bytes to be sent with respect to the previously matched chunks (chunks the source knows to be present on the target).

4. RESULTS

We used FTP to download a total of 7 male human genomes for our experiments: 2 from the KOREF dataset [2] (sequence 0090224 and 20090931) and 5 from the UCSC genome browser [13] (sequence hg15 through hg19). Each human genome consists of 25 sequence files in FASTA format: 22 for chromosomes 1 through 22, 1 X chromosome, 1 Y chromosome, and 1 mitochondrial DNA sequence (M). The total size of a single uncompressed human genome in this format is 3.0GB. All of the genomes were available as compressed *gzipped* chromosomes, except for one of the KOREF sequences that was only available as uncompressed chromosomes. The FASTA format is meant to be human readable and encodes each base-pair as an ASCII character (“G”, “A”, “C”, “T”), with a newline every fixed set of characters (60 in the KOREF dataset, 50 in the hg*) for human readability. Additional characters may encode metadata such as “>” for comments, or characters such as “N” that represent any nucleic acid.

4.1 Genome Redundancy Measurements

The first experiment we conducted quantifies the amount of similarity across the set of 7 genomes and within each genome, using fixed sized non-overlapping chunks from 4KB to 100KB. We split every chromosome of size c into the chunks of size s , where bytes $[s(i-1), \min(si, c))$ compose chunk i . Every chunk was md5 hashed and inserted into a hash table shared across the 7 genomes.

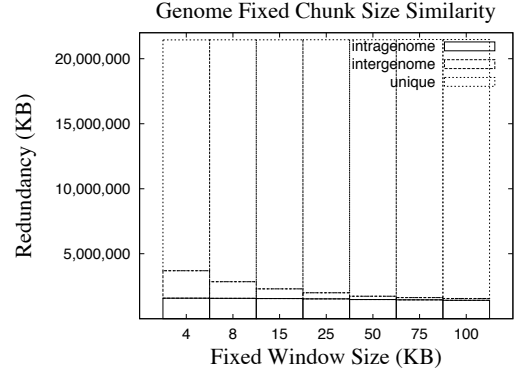


Figure 2: Similarity across 7 human genomes with fixed sized chunks.

Fig. 2 shows the amount of storage space that would be saved using a chunk cache with the specified window size. The legend is as follows: **unique** is the number of unique hashes multiplied by the chunk size; **intra-genome** is the sum of the number of redundant hashes multiplied by the chunk size, within each human genome; **intergenome** is the total number of redundant hashes multiplied by the chunk size, and subtracted by the total **intra-genome** size. Hence, **intergenome + intra-genome** is the total amount of redundant chunks multiplied by the chunk size; we use this metric to quantify the amount of redundant bytes.

We expected a larger amount of redundancy across the set of 7 genomes. In the best case (4KB chunks), only 3.7GB of the 21.5GB were redundant. However, these results do confirm our intuition that an intergenome approach offers a greater opportunity for exploiting redundancy; indeed, the amount of intra-genome redundancy remained fixed across all chunk sizes, while the intergenome redundancy increased rapidly with smaller chunks. This rapid growth suggests that much of the redundancy in larger chunks is masked by small differences. Recall that an individual byte difference within the chunk results in a different hash.

To test whether the alignment of the windows affects redundancy, we ran the same 7 genome experiment using Rabin fingerprints. We computed the fingerprint of every 48 byte window of each chromosome; if the lower k bits of this fingerprint is 0 then we create a new chunk boundary. We added the constraint that no chunk is less than 4KB in size, but we did not impose an upper bound.

Fig. 3 demonstrates log-log graph of the chunk sizes when using $k = 18$, with an expected chunk size of 256KB. This graph is similar to that in [12] which analyzed the Emacs-20.7 source tree with 2KB expected chunk size, although some differences are apparent. Primarily, the size of the chunks are much less uniform and the distribution is skewed towards larger chunk sizes. Although 90% of the chunks are less than 1MB, there are several outliers with very large chunk sizes. For example, the largest 1% of the chunks con-

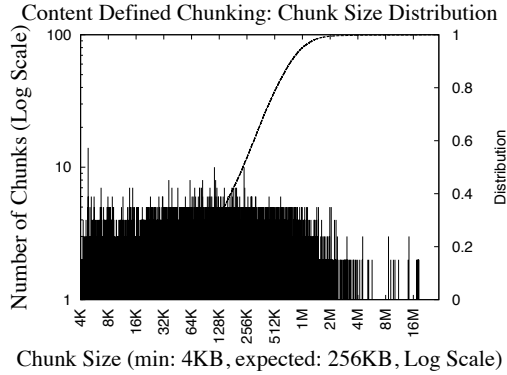


Figure 3: CDC size log-log distribution across 7 genomes, with $k = 18$. The left axis is a histogram of the chunk sizes, while the right axis measures the CDF of the chunk sizes.

tained 6% of the total bytes; 5 chunks were greater than 31MB the largest chunk was 31.75MB.

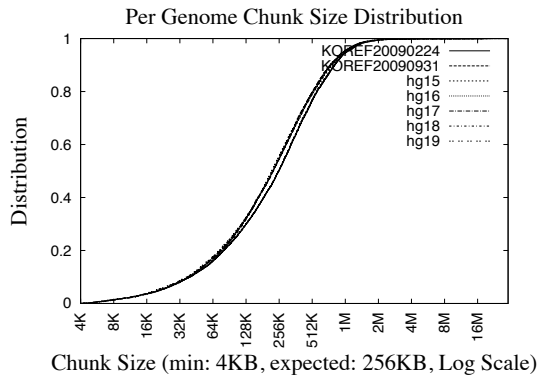


Figure 4: CDC size distribution for each individual genome, with $k = 18$.

Fig. 4 examines the CDC size CDF for each individual genome. Unsurprisingly, it is very similar to that of 3, however two separate bands emerge: the band to the left contains the entire hg* dataset and the band on right contains the KOREF dataset. These results suggest that genomes share a similar structure, however genomes from different datasets have a slightly different structure. Future work will investigate the role of the CDC size CDF as a mechanism for identifying similar chunks prior to computing deltas.

4.2 Genome Paired Deltas

In an experiment similar to [24], we took the two genomes from the KOREF dataset, split them into fixed sized chunks of 20MB, and computed deltas of each corresponding chunk using the **bsdiff** [18] and **xdelta** [23] binary delta compression tools.

sion tools.

Briefly, **xdelta** is based on the algorithm for **rsync** and generates standardized VCDIFF output [14]. **xdelta** and **rsync** use a “two signature” algorithm where a quick, collision susceptible signature (hash) is computed on a rolling window over every byte offset, and a computationally expensive but collision resilient hash is used if two chunks have the same weak signature (to ensure that they are the same chunk).

bsdiff uses an instruction set composed of copy and insert operations and is designed to address the “pointer problem” found in similar versions of executable code: it is likely that addresses will change in the same way throughout the executable. This tool exploits the pointer phenomena by recording the first case of an address substitution and predicting future substitutions (without recording them).

Fig. 5 lists the compression sizes in comparison with **gzip** and **GRS** from [24]. **GRS** uses a similar technique of splitting each chromosome into chunks and computing pair-wise deltas, however they apply a windowing algorithm to discover better chunks boundaries, and they use a modified version of **diff** to compute the deltas. For this experiment, we used KOREF 20090224 as the reference and computed deltas for KOREF 20090931¹. After generating the patch file, we applied the patch to the reference and verified that the output is correct—no errors occurred throughout the experiment. A total of 162 patch files were generated. The total duration for each tool (including the validation) was 1512 seconds (25.2 minutes) for **xdelta** and 9324 seconds (2.59 hours) for **bsdiff** on a modest single core 2.2GHz AMD 64 computer with 2GB of RAM. For both tools, applying the patch to construct the reference chunk never exceeded 3 seconds.

The results in Fig. 5 demonstrate that unaltered, traditional binary data differencing algorithms can be extremely effective for genomic delta compression in some scenarios, a surprising result considering that such tools are designed for compressing binary executables. Although **bsdiff** takes several hours to perform the compression, this delay is not discouraging because the process can be parallelized by the number of chunks (162 in this case), and cached deltas can void redundant computations. Such caching allows the user to balance CPU, network bandwidth, and storage utilization. Future work will investigate the biological explanations as to why **bsdiff** performs so well in this scenario. It is conceivable that a phenomena analogous to the “pointer problem” is occurring.

Fig. 6 lists the compression results for all $\binom{7}{2} = 21$ pair-wise deltas, with 20MB fixed length chunks. Although all of the compressed files are smaller than their **gzipped** counterparts, the compression sizes are an order of magnitude larger than those in Fig. 5; thus, these tools are not a panacea. This table also shows that the binary delta tools can exploit redundancy better for pairs of genomes from the same

¹ We realized later that in [24] the reference and is delta swapped; we did not have enough time to repeat the experiment to provide a proper comparison, however future work will examine the relationship between achieved compression rates when swapping the reference and the delta

chromosome	raw size	gzip	xdelta	GRS	bsdifff
chr01.fa	240MB	67M	1.9MB	1.3MB	756KB
chr02.fa	236MB	71M	1.9MB	1.3MB	760KB
chr03.fa	194MB	58M	1.5MB	987KB	552KB
chr04.fa	186MB	56M	1.7MB	1.1MB	624KB
chr05.fa	176MB	53M	1.4MB	965KB	552KB
chr06.fa	166MB	50M	1.3MB	885KB	496KB
chr07.fa	154MB	46M	1.6MB	1.0MB	636KB
chr08.fa	142MB	43M	1.1MB	764KB	412KB
chr09.fa	137MB	36M	1.3MB	844KB	496KB
chr10.fa	132MB	40M	1.1MB	750KB	432KB
chr11.fa	131MB	39M	1.1MB	738KB	424KB
chr12.fa	129MB	39M	1MB	686KB	384KB
chr13.fa	111MB	29M	692KB	508KB	280KB
chr14.fa	104MB	27M	680KB	473KB	264KB
chr15.fa	98MB	25M	724KB	485KB	288KB
chr16.fa	87MB	24M	860KB	555KB	332KB
chr17.fa	77MB	23M	768KB	494KB	300KB
chr18.fa	74MB	23M	536KB	399KB	228KB
chr19.fa	62MB	17M	676KB	390KB	228KB
chr20.fa	61MB	18M	400KB	276KB	152KB
chr21.fa	46MB	11M	284KB	221KB	132KB
chr22.fa	49MB	11M	316KB	256KB	160KB
chrM.fa	20KB	8.0KB	80KB	17KB	4.0KB
chrX.fa	151MB	47M	4.7MB	3.1MB	1.8MB
chrY.fa	57MB	7.8M	784KB	579KB	376KB
TOTAL	3GB	850M	28MB	18.8MB	11MB

Figure 5: Compression results for the KOREF dataset.

	KOREF*	hg*	KOREF*×hg*
bsdifff	11MB	217.6 ± 45.47MB	625.7 ± 11MB

Figure 6: Average (and standard deviation) compression size for all pairwise combinations of genome deltas. Sizes are separated by their respective datasets. The delta compression tools perform worse when compressing two genomes from separate datasets.

dataset, while they have difficulty with genomes from separate datasets, as foreshadowed by Fig. 4. Future work hopes to explain why the tools perform so well in the KOREF dataset versus the hg*.

5. CONCLUSION

We identified the pending genomic data network problem and offered a survey of current approaches from the fields of bioinformatics and operating systems research. We argue that a distributed approach that addresses the problem by exploiting intergenome redundancies can greatly reduce the amount of data that is sent. In one experiment, a naïve application of binary delta software gave compression rates that are better than previously published results. This preliminary work serves as the first step towards the development of a tool that can exploit such redundancies automatically.

6. REFERENCES

- [1] Bio-mirror: biosequence & bioinformatics data. <http://www.bio-mirror.net/>, December 2011.
- [2] S. Ahn, T. Kim, S. Lee, et al. The first korean genome sequence and analysis: Full genome sequencing for a socio-ethnic group. In *Genome Research*, 2009.
- [3] W. Allcock, J. B. R. Kettimuthu, M. Link, C. Dumitrescu, I. Raicu, and I. Foster. The globus striped gridftp framework and server. In *ACM/IEEE conference on Supercomputing*, 2005.
- [4] M. Brandon, D. Wallace, and P. Baldi. Data structures and compression algorithms for genomic sequence data. In *Bioinformatics 25*, pages 1731–1738, 2009.
- [5] S. Christley, Y. Lu, C. Li, and X. Xie. Human genomes as email attachments. In *Bioinformatics 25*, pages 274–275, 2008.
- [6] T. Cover and J. Thomas. *Elements of information theory*. Wiley-Interscience; 2nd edition, Hoboken, New Jersey, U.S., 2006.
- [7] R. Durbin and . G. Consortium. A map of human genome variation from population-scale sequencing. In *Nature 467*, pages 1061–1073, 2010.
- [8] M. H.-Y. Fritz, R. Leinonen, G. Cochrane, and E. Birney. Efficient storage of high throughput sequencing data using reference-based compression.
- [9] Y. Gu and R. Grossman. Udt: Udp-based data transfer for high-speed wide area networks. In *Computer Networks. Volume 51, Issue 7*.
- [10] K. Harmon. Genome sequencing for the rest of us. In *Scientific American*, 2010.
- [11] L. Heath, A. Hou, H. Xia, and L. Zhang. A genome compression algorithm supporting manipulation. In *Proceedings of the 9th International Conference on Computational Systems Bioinformatics*, pages 38–49, 2010.
- [12] N. Jain, M. Dahlin, and R. Tewari. Taper: Tiered approach for eliminating redundancy in replica synchronization. In *In USENIX Conference on File and Storage Technologies*, pages 219–228, 2005.
- [13] W. Kent, C. Sugnet, T. Furey, K. Roskin, T. Pringle, A. Zahler, and D. Haussler. The human genome browser at ucsc. In *Genome Research*, pages 996–1006, 2002.
- [14] D. Korn and K. Vo. Engineering a differencing and compression data format. In *In USENIX Annual Technical Conference, General Track*, pages 219–228, 2002.
- [15] H. Li, B. Handsaker, A. Wysoker, T. Fennell, J. Ruan, N. Homer, G. Marth, G. Abecasis, and R. Durbin. The sequence alignment/map format and samtools. In *Bioinformatics 25*, pages 2078–2079, 2009.
- [16] U. Manber. Finding similar files in a large file system. In *Proceedings of the USENIX Winter Technical Conference*, pages 1–10, 1994.
- [17] A. Muthitacharoen, B. Chen, and D. Mazieres. A low-bandwidth network file system. In *Proceedings of the 18th ACM Symposium on Operating Systems Principles*, pages 174–187, 2001.
- [18] C. Percival. *Matching with mismatches and assorted applications*. PhD thesis, University of Oxford, 2006.
- [19] H. Pucha, David, Andersen, and M. Kaminsky. Exploiting similarity for multi-source downloads using file handprints. In *Networked System Design and Implementation*, pages 15–28, 2007.
- [20] M. Punniyakotti and S. Wood. Large genomic data transfer methods. Technical report, UC Santa Cruz,

- 2011.
- [21] M. Rabin. Fingerprinting by random polynomials. Technical report, Harvard University, 1981.
 - [22] R. Service. The race for the \$1000 genome. *Science Vol. 311*, 2006.
 - [23] A. Tridgell. *Efficient Algorithms for Sorting and Synchronization*. PhD thesis, The Australian National University, 1999.
 - [24] C. Wang and D. Zhang. A novel compression tool for efficient storage of genome resequencing data. nucleic acids research. In *Nucleic Acids Research 39*, 2011.

APPENDIX

A. DEFINITIONS

In order to accurately discuss data compression techniques, we use the following (albeit non-standard) definitions² loosely based on [14]. A data compression strategy consists of a pair of algorithms for encoding and decoding strings S ($A_{enc} : S \rightarrow S, A_{dec} : S \rightarrow S$) (a target file) such that $A_{enc}(T) = C \wedge A_{dec}(C) = T$ where T and C are finite strings drawn from a finite alphabet Σ^* (a binary representation of the target file). Much previous work has focused on finding strategies that minimizing $|C|$ for all strings S (i.e. LZW, Huffman, Golomb codes) using string matching algorithms to find large, frequently appearing substrings that are replaced by smaller strings (codes) in S during encoding. Unix's `gzip` program is a popular data compression tool. In the context of a data transfer where Alice wishes to send a file S to Bob, we assume Alice knows A_{enc} and Bob knows A_{dec} , and we wish to minimize $|A_{enc}(T)|$.

Since the computational complexity of A_{enc} is typically non-linear and memory intensive, T is partitioned into n smaller contiguous subsequences (t_i) or target windows, which are compressed separately to increase compression speed at the expense of reduced compression. In the simplest strategy, if we denote $\sum$ for concatenation:

$$\begin{aligned} A_{enc}(T) &= \sum_{i=1}^n A_{enc}(t_i) = \sum_{i=1}^n c_i = C \\ A_{dec}(C) &= \sum_{i=1}^n A_{dec}(c_i) = \sum_{i=1}^n t_i = T \\ \forall i \in \{1, \dots, n\} &(t_i = A_{dec}(A_{enc}(t_i))) \end{aligned}$$

Note that A_{enc} and A_{dec} must agree on some protocol to delimit the windows, and windows can either be of fixed or variable length.

To further increase compression, the construction of a target window may be a function of a source window which can either belong to a different source file, or belong to the target file. Data differencing strategies $A_{enc}(T, S) = C, A_{dec}(T, S)$ compute the differences between a source file S and target file T such that the target file T can be rebuilt from C and S . C is typically composed of a sequence of instructions from a determined instruction set, such as insert, delete and

²We did not find a consistent set of definitions across his literature survey.

edit. These instructions are then executed by the decoder to rebuild T . Unix's `diff` (A_{enc}) and `patch` (A_{dec}) programs compose a popular and simple data differencing tool, where C is known as a patch file. In the simplest strategy:

$$\begin{aligned} A_{enc}(T, S) &= \sum_{i=1}^n A_{enc}(t_i, s_i) = \sum_{i=1}^n c_i = C \\ A_{dec}(C, S) &= \sum_{i=1}^n A_{dec}(c_i, s_i) = \sum_{i=1}^n t_i = T \\ \forall i \in \{1, \dots, n\} &(t_i = A_{dec}(A_{enc}(t_i, s_i), s_i)) \end{aligned}$$

Note that in both cases the dependent source window s_i must be available at the time of decoding window c_i in order to accurately reconstruct the target window. Also, c_i can be defined in terms of an already compressed window t_k (where $k < i$ in this simple strategy).³ From the data transfer context, we wish to minimize $|A_{enc}(T, S)|$ given the constraint that Alice knows A_{enc} and S , and Bob knows A_{dec} and S . Algorithms that match an ideal source window s_i to a given target window t_i are called windowing algorithms, while algorithms that unify data compression with data differencing are known as delta compression algorithms.

³ There has been much research in decoupling A_{enc} from A_{dec} so that one can use different encoding algorithms with the same decoding algorithm, such that,

$$\forall i (A_{enc_i}(T, S) = C_i \wedge A_{dec}(C_i, S) = T)$$

These approaches abstract a common instruction set and execution environment (such as defining an instruction cache) for encoding and decoding and express T as instructions for this environment; the decoder essential becomes an interpreter that executes the delta instructions on a particular reference.